

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a stylized tree structure.

Number Representation

COMP211 Connor McMahon

Number Representation

- **Remember, bits are just bits!** We can't determine their value until we know the representation used to encode them.
- Representations we have learned so far
 - Unsigned
 - One's Complement
 - Two's Complement

When is each representation used?

- **One's complement**

- Never used in practice anymore
- Mainly taught as a steppingstone towards understanding two's complement

- **Two's complement**

- Widely used for representing signed integers

- **Unsigned**

- Used only when non-negative values are needed
- Allows for a large positive range than two's complement with the same bit width

Representations Summary

What is the value of the following binary numbers interpreted in unsigned, one's complement, and two's complement notation?

Q1. 1111_2

unsigned:

1's complement:

2's complement:

Q2. 0101_2

unsigned:

1's complement:

2's complement:

Q3. 1010_2

unsigned:

1's complement:

2's complement:

Representations Summary



What is the decimal value of the following binary numbers interpreted in unsigned, one's complement, and two's complement notation?

Q1. 1111_2

unsigned: 15

1's complement: -0

2's complement: -1

Q2. 0101_2

unsigned: 5

1's complement: 5

2's complement: 5

Q3. 1010_2

unsigned: 10

1's complement: -5

2's complement: -6

Ranges

- What range of values can we represent in 3-bit unsigned binary?
 - Range:
 - Smallest number:
 - Largest number:
- What range of values can we represent in 3-bit 2's complement binary?
 - Range:
 - Smallest number:
 - Largest number:

Ranges



- What range of values can we represent in 3-bit unsigned binary?
 - Range: [0, 7]
 - Smallest number: 0b000
 - Largest number: 0b111
- What range of values can we represent in 3-bit two's complement binary?
 - Range: [-4, 3]
 - Smallest number: 0b100
 - Largest number: 0b011

Generalizing Ranges

Unsigned Range: $[0, 2^n - 1]$

Two's Complement Range: $[-(2^{n-1}), 2^{n-1} - 1]$

Two's Complement Range

- With n bits, we can represent 2^n unique values
- Split evenly, this yields

Two's Complement Range

- With n bits, we can represent 2^n unique values
- Split evenly, this yields
 - $2^n/2 = 2^{n-1}$ positive values
 - $2^n/2 = 2^{n-1}$ negative values
- But we also need to include 0
- To make room for 0, we sacrifice one positive value
- Final Counts
 - $2^n/2 - 1 = 2^{n-1} - 1$ positive values
 - $2^n/2 = 2^{n-1}$ negative values
- Range: $[-(2^{n-1}), 2^{n-1}-1]$

Binary Addition

Binary Addition

Decimal

$$\begin{array}{r} 12 \\ + 8 \\ \hline \end{array}$$

Binary

$$\begin{array}{r} 1011_2 \\ + 0011_2 \\ \hline \end{array}$$

Binary Addition

Decimal

$$\begin{array}{r} 1 \\ + 12 \\ 8 \\ \hline 20 \end{array}$$

Binary

$$\begin{array}{r} 11 \\ + 1011_2 \\ 0011_2 \\ \hline 1110 \end{array}$$

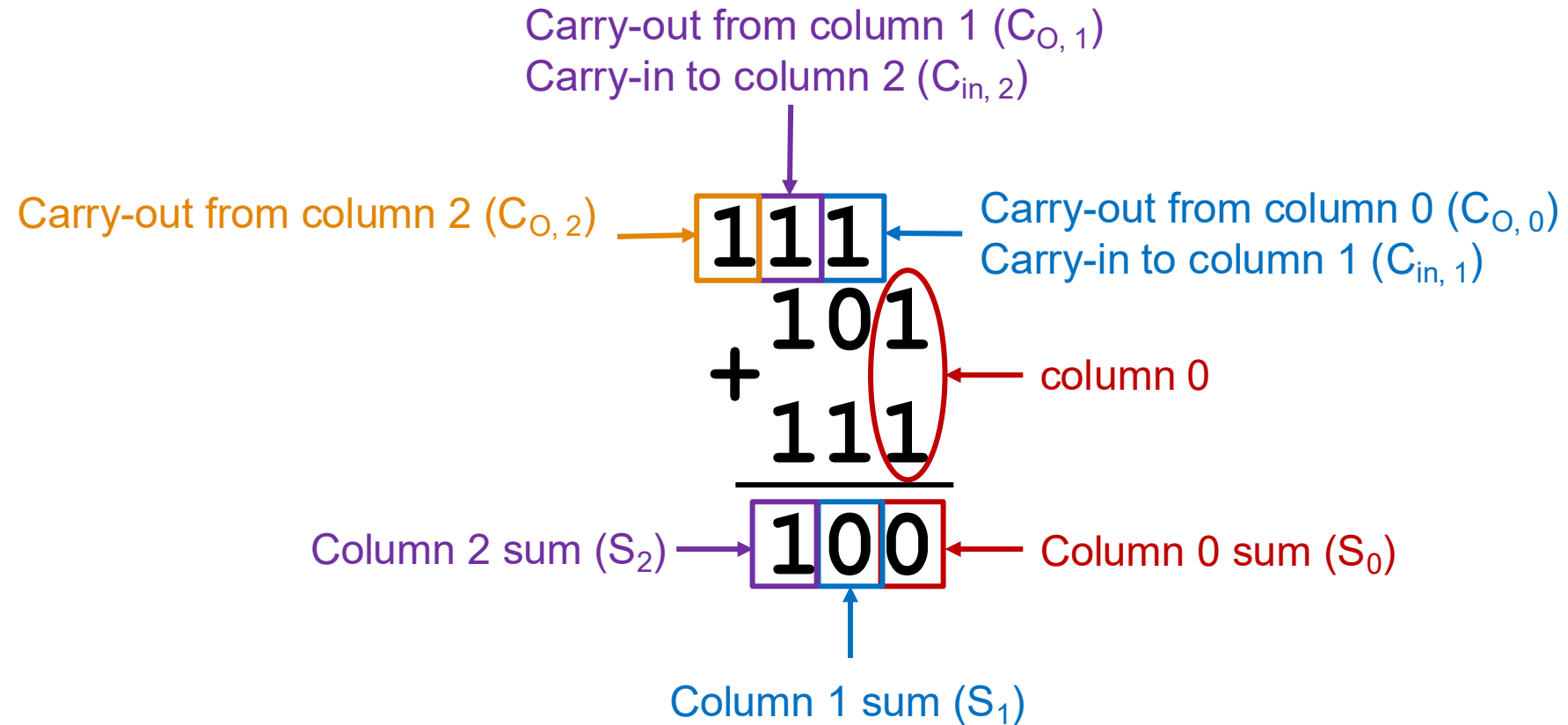
Binary Addition Terminology

$$\begin{array}{r} + 101 \\ 111 \\ \hline \end{array}$$

Binary Addition Terminology

$$\begin{array}{r} 111 \\ + 101 \\ + 111 \\ \hline 100 \end{array}$$

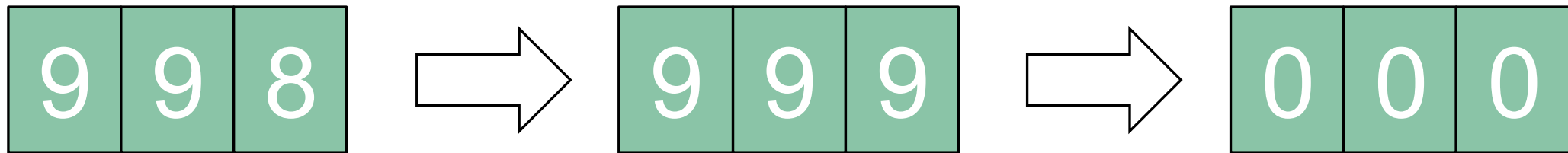
Binary Addition Terminology



Overflow

Overflow in the Physical World

- Imagine a speedometer with only 3 digits.
 - The range of values it can display is [000, 999]
 - If you're at 999 and increase by 1, it doesn't show 1000 (because there are only 3 digits)
 - Instead, it wraps back around to 000
- This “wrapping around” is exactly what happens with integer overflow in binary numbers!



Binary Addition: Unsigned Overflow

- **Overflow** occurs when the result of the operation cannot be represented with the given number of bits.
 - When an operation overflows, the result is incorrect.

$$\begin{array}{r} + 11 \\ 5 \\ \hline 16 \end{array}$$

$$\begin{array}{r} + 1011_2 \\ 0101_2 \\ \hline \end{array}$$

Binary Addition: Unsigned Overflow

- **Overflow** occurs when the result of the operation cannot be represented with the given number of bits.
 - When an operation overflows, the result is incorrect.

$$\begin{array}{r} + 11 \\ 5 \\ \hline 16 \end{array}$$

If the final result is limited to 4 bits, this 1 has no place to be stored.

$$\begin{array}{r} + 1011_2 \\ 0101_2 \\ \hline 0000 \end{array}$$

(A red circle highlights the leading '1' in the result '1111', with an arrow pointing to the explanatory text.)

In unsigned addition, overflow occurs when the carry-out of the last column is 1.

Unsigned Binary Addition and Overflow

- Determine the 4-bit result of the following unsigned operations. Did an overflow occur? (Yes/No)

$$\begin{array}{r} 1010_2 \\ + 0101_2 \\ \hline \end{array}$$

$$\begin{array}{r} 1001_2 \\ + 0111_2 \\ \hline \end{array}$$

$$\begin{array}{r} 1010_2 \\ + 0111_2 \\ \hline \end{array}$$

$$\begin{array}{r} 0110_2 \\ + 0110_2 \\ \hline \end{array}$$

Unsigned Binary Addition and Overflow



- Determine the 4-bit result of the following unsigned operations. Did an overflow occur? (Yes/No)

$$\begin{array}{r} 1010_2 \\ + 0101_2 \\ \hline 1111 \end{array} \quad \text{no}$$

$$\begin{array}{r} 1111 \\ 1001_2 \\ + 0111_2 \\ \hline 0000 \end{array} \quad \text{yes}$$

$$\begin{array}{r} 111 \\ 1010_2 \\ + 0111_2 \\ \hline 0001 \end{array} \quad \text{yes}$$

$$\begin{array}{r} 11 \\ 0110_2 \\ + 0110_2 \\ \hline 1100 \end{array} \quad \text{no}$$

Two's Complement Addition

Two's Complement Addition

- Arithmetic in 2's complement works the same as unsigned

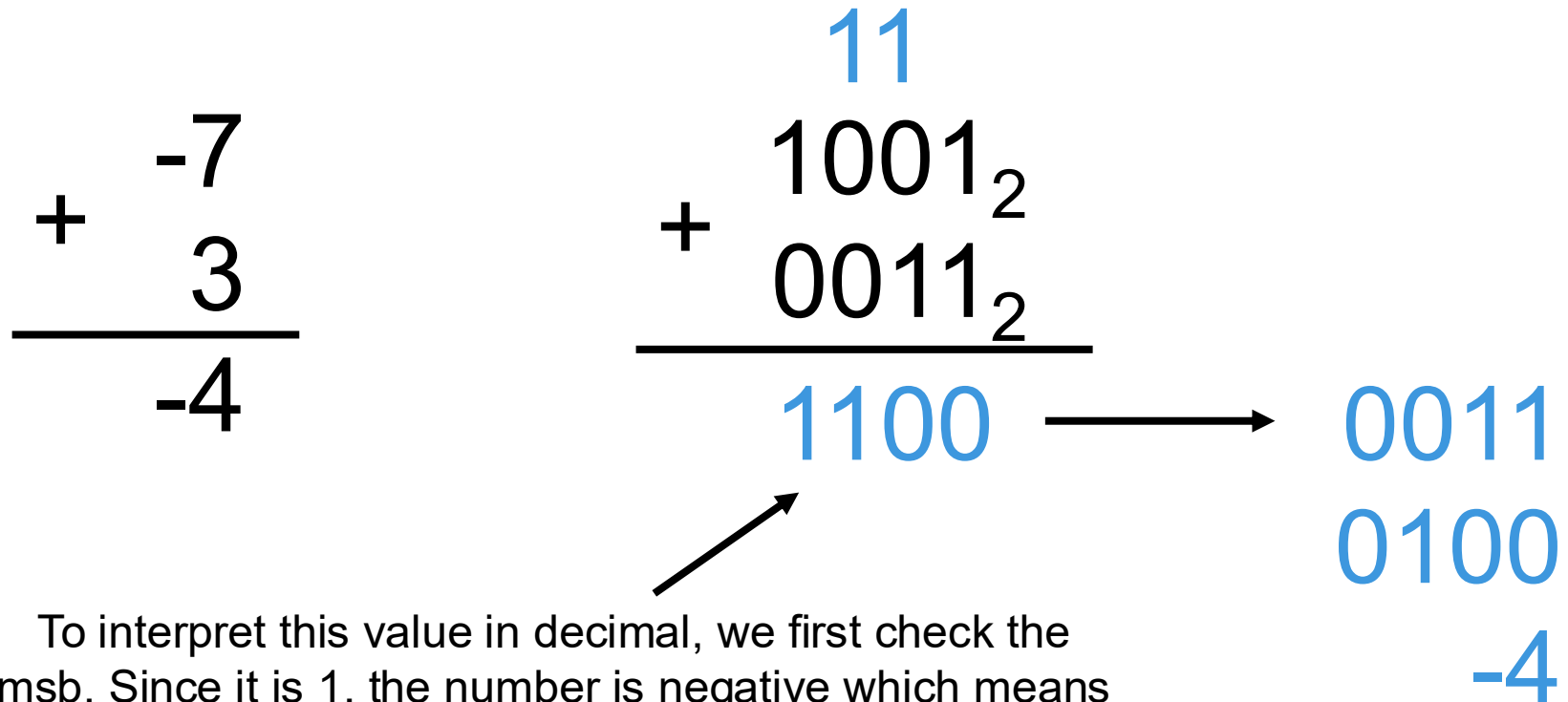
$$\begin{array}{r} + \quad -7 \\ \quad 3 \\ \hline -4 \end{array}$$

$$\begin{array}{r} + \quad 1001_2 \\ \quad 0011_2 \\ \hline \end{array}$$

Two's Complement Addition

- Arithmetic in 2's complement works the same as unsigned

$$\begin{array}{r} + \quad -7 \\ \quad 3 \\ \hline -4 \end{array}$$
$$\begin{array}{r} \\ + \quad 1001_2 \\ \quad 0011_2 \\ \hline 1100 \end{array} \longrightarrow \begin{array}{r} 0011 \\ 0100 \\ -4 \end{array}$$



To interpret this value in decimal, we first check the msb. Since it is 1, the number is negative which means that to find its decimal value, we flip the bits and add 1.

Two's Complement Overflow

- **Overflow** occurs when the result of the operation cannot be represented with the given number of bits.

Adding	Overflow occurs when
two positive numbers	the result is negative
two negative numbers	the result is positive
two numbers of opposite signs	will never occur

Two's Complement Overflow – your turn

All answers must be 4 bits

Determine the 4-bit result of the following 2's complement operations. Did an overflow occur? (Yes/No)

Q1

$$\begin{array}{r} 0111_2 \\ + 0101_2 \\ \hline \end{array}$$

Q2

$$\begin{array}{r} 1111_2 \\ + 0110_2 \\ \hline \end{array}$$

Q3

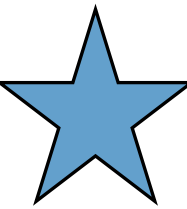
$$\begin{array}{r} 1000_2 \\ + 1001_2 \\ \hline \end{array}$$

Q4

$$\begin{array}{r} 1101_2 \\ + 1011_2 \\ \hline \end{array}$$

Two's Complement Overflow – your turn

All answers must be 4 bits



Determine the 4-bit result of the following unsigned operations. Did an overflow occur? (Yes/No)

Q1

$$\begin{array}{r} 111 \\ + 0111_2 \\ + 0101_2 \\ \hline 1100 \rightarrow -4 \end{array} \quad \begin{array}{r} 7 \\ + 5 \\ \hline 12 \end{array}$$

Overflow

Q2

$$\begin{array}{r} 111 \\ + 1111_2 \\ + 0110_2 \\ \hline 0101 \rightarrow 5 \end{array} \quad \begin{array}{r} -1 \\ + 6 \\ \hline 5 \end{array}$$

No
Overflow

Q3

$$\begin{array}{r} 1 \\ + 1000_2 \\ + 1001_2 \\ \hline 0001 \rightarrow 1 \end{array} \quad \begin{array}{r} -8 \\ + -7 \\ \hline -15 \end{array}$$

Overflow

Q4

$$\begin{array}{r} 1111 \\ + 1101_2 \\ + 1011_2 \\ \hline 1000 \rightarrow -8 \end{array} \quad \begin{array}{r} -3 \\ + -5 \\ \hline -8 \end{array}$$

No
Overflow

Unsigned Addition

$$\begin{array}{r} + 111_2 \\ 010_2 \\ \hline \end{array}$$

2's Complement Addition

$$\begin{array}{r} + 111_2 \\ 010_2 \\ \hline \end{array}$$

Unsigned Addition

$$\begin{array}{r} 11 \\ + 111_2 \\ \hline 010_2 \end{array} \qquad \begin{array}{r} 7 \\ + 2 \\ \hline 9 \end{array}$$

This is not 9! $\rightarrow$ 001

There is a carry-out of the last column, so overflow has occurred.

Remember, overflow means the result is incorrect. To prove to ourselves that overflow did occur, we can check the math in decimal. If we interpret the result as an unsigned value, the result is incorrect. It is not possible to represent the number 9 with only 3 bits.

2's Complement Addition

$$\begin{array}{r} 11 \\ + 111_2 \\ \hline 010_2 \end{array} \qquad \begin{array}{r} -1 \\ + 2 \\ \hline 1 \end{array}$$

We are adding numbers of opposite signs, so overflow is not possible.

Remember, no overflow means the result is correct. To prove to ourselves that overflow did not occur, we can check the math in decimal. If we interpret the result as a 2's complement value, it is correct. We have enough bit to represent the result, 1.

Unsigned Addition

$$\begin{array}{r} + 011_2 \\ 010_2 \\ \hline \end{array}$$

2's Complement Addition

$$\begin{array}{r} + 011_2 \\ 010_2 \\ \hline \end{array}$$

Unsigned Addition

$$\begin{array}{r} 1 \\ + 011_2 \\ + 010_2 \\ \hline 101 \end{array} \qquad \begin{array}{r} 3 \\ + 2 \\ \hline 5 \end{array}$$

Overflow did not occur because the carry-out of the last column is 0.

Remember, no overflow means the result is correct. To prove to ourselves that overflow did not occur, we can check the math in decimal. If we interpret the result as an unsigned value, the result is correct. There are enough bits to represent the result, 5.

2's Complement Addition

$$\begin{array}{r} 1 \\ + 011_2 \\ + 010_2 \\ \hline 101 \end{array} \qquad \begin{array}{r} 3 \\ + 2 \\ \hline 5 \end{array}$$

This is not 5! It is -3! →

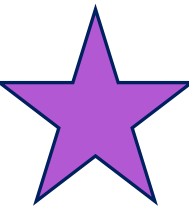
Adding two positive values produced a negative result, so overflow occurred.

Remember, overflow means the result is incorrect. To prove to ourselves that overflow did occur, we can check the math in decimal. If we interpret the result as a 2's complement value, it is incorrect. We cannot represent 5 with only 3 bits.

Overflow

- Before 2014, YouTube used a 32-bit signed integer to track video views. What was the maximum number of views this counter could record? Signed integers are stored in 2's complement format. First, place your answer as a power of 2. Then use a calculator to compute the value.
- Trivia: Which video caused YouTube to update the view counter to 64 bits?

Overflow



- Before 2014, YouTube used a 32-bit signed integer to track video views. What was the maximum number of views this counter could record? Signed integers are stored in 2's complement format.
 - $2^{32-1}-1 = 2^{31}-1 = 2,147,483,647$ views (2.15 billion)
- The counter is now safe until a video reaches
 - 9,223,372,036,854,780,000 – 9.2 billion billion views



Two's Complement Subtraction

Two's Complement Subtraction

To perform subtraction in 2's complement, negate the second operand, then add it to the first operand.

$$\begin{array}{r} -10 \\ -4 \\ \hline -14 \end{array} \quad \begin{array}{r} 10110_2 \\ -00100_2 \\ \hline \end{array} == \begin{array}{r} 10110_2 \\ +11100_2 \\ \hline \end{array} \quad \begin{array}{r} -10 \\ + -4 \\ \hline -14 \end{array}$$

Two's Complement Subtraction

To perform subtraction in 2's complement, negate the second operand, then add it to the first operand.

$$\begin{array}{r} -10 \\ -4 \\ \hline -14 \end{array} \quad \begin{array}{r} 10110_2 \\ -00100_2 \\ \hline 10010_2 \end{array} == \begin{array}{r} 10110_2 \\ +11100_2 \\ \hline 10010_2 \end{array} \quad \begin{array}{r} -10 \\ + -4 \\ \hline -14 \end{array}$$

Two's Complement Subtraction

To perform subtraction in 2's complement, negate the second operand, then add it to the first operand.

$$\begin{array}{r} -10 \\ - -4 \\ \hline -6 \end{array} \quad \begin{array}{r} 10110_2 \\ - 11100_2 \\ \hline 11010_2 \end{array} == \begin{array}{r} 10110_2 \\ + 00100_2 \\ \hline \end{array} \quad \begin{array}{r} -10 \\ + 4 \\ \hline -6 \end{array}$$

Two's Complement Subtraction

To perform subtraction in 2's complement, negate the second operand, then add it to the first operand.

$$\begin{array}{r} -10 \\ - -4 \\ \hline -6 \end{array} \quad \begin{array}{r} 10110_2 \\ - 11100_2 \\ \hline 11010_2 \end{array} == \begin{array}{r} 10110_2 \\ + 00100_2 \\ \hline 11010_2 \end{array} \quad \begin{array}{r} -10 \\ + 4 \\ \hline -6 \end{array}$$

Two's Complement Subtraction – your turn

All answers must be 5 bits

Indicate whether overflow occurred.

Q1

$$\begin{array}{r} 00101_2 \\ - 00111_2 \\ \hline \end{array}$$

Q2

$$\begin{array}{r} 10101_2 \\ - 10110_2 \\ \hline \end{array}$$

Q3

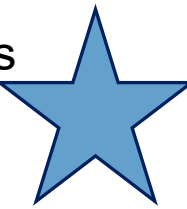
$$\begin{array}{r} 11111_2 \\ - 01000_2 \\ \hline \end{array}$$

Q4

$$\begin{array}{r} 01110_2 \\ - 11100_2 \\ \hline \end{array}$$

Two's Complement Subtraction – your turn

All answers must be 5 bits



Q1

$$\begin{array}{r} 00101_2 \\ - 00111_2 \\ \hline \end{array}$$

$$\begin{array}{r} \textcolor{red}{1} \\ + 00101_2 \\ + 11001_2 \\ \hline \textcolor{red}{11110}_2 \end{array}$$

No Overflow

Q2

$$\begin{array}{r} 10101_2 \\ - 10110_2 \\ \hline \end{array}$$

$$\begin{array}{r} + 10101_2 \\ + 01010_2 \\ \hline \textcolor{red}{11111}_2 \end{array}$$

No Overflow

Q3

$$\begin{array}{r} 11111_2 \\ - 01000_2 \\ \hline \end{array}$$

$$\begin{array}{r} \textcolor{red}{11} \\ + 11111_2 \\ + 11000_2 \\ \hline \textcolor{red}{10111}_2 \end{array}$$

No Overflow

Q4

$$\begin{array}{r} 01110_2 \\ - 11100_2 \\ \hline \end{array}$$

$$\begin{array}{r} \textcolor{red}{11} \\ + 01110_2 \\ + 00100_2 \\ \hline \textcolor{red}{10010}_2 \end{array}$$

Overflow

Two's Complement in C

How Numbers Are Stored in Your Programs

- Signed numbers are encoded using **2's complement**
- Unsigned numbers are encoded using **unsigned binary**
- For example, let's store -1 as an 8-bit number
 - The value would be stored at **0b1111_1111**.
 - If interpreted as a **signed** number, it represents **-1**.
 - If interpreted as an **unsigned** number, it represents **255**.

What does this program print?

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int8_t num = 67;
    int8_t neg_num = -67;

    printf("Integer Representations (Positive):\n");
    printf("Signed: %d\n", num);
    printf("Unsigned: %u\n", (uint8_t)num);
    printf("ASCII: %c\n\n", num);

    printf("Integer Representations (Negative):\n");
    printf("Signed: %d\n", neg_num);
    printf("Unsigned: %u\n", (uint8_t)neg_num);
    printf("ASCII: %c\n", neg_num);
    return EXIT_SUCCESS;
}
```

Integer Representations (Positive):

Signed:

Unsigned:

ASCII:

Integer Representations (Negative):

Signed:

Unsigned:

ASCII:

Full ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

What does this program print?



```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int8_t num = 67;
    int8_t neg_num = -67;

    printf("Integer Representations (Positive):\n");
    printf("Signed: %d\n", num);
    printf("Unsigned: %u\n", (uint8_t)num);
    printf("ASCII: %c\n\n", num);

    printf("Integer Representations (Negative):\n");
    printf("Signed: %d\n", neg_num);
    printf("Unsigned: %u\n", (uint8_t)neg_num);
    printf("ASCII: %c\n", neg_num);
    return EXIT_SUCCESS;
}
```

Integer Representations (Positive):

Signed: 67

Unsigned: 67

ASCII: C

Integer Representations (Negative):

Signed: -67

Unsigned: 189

ASCII: ?

Signed: 67 = 0b0100_0011

Signed: -67 = 0b1011_1101

Unsigned: 0b1011_1101 = 189